

Experiment-1: Image Resizing, Cropping and Rotation

Program

```
# Install OpenCV
!pip install opencv-python

# Import libraries
import cv2
from google.colab import files
from google.colab.patches import cv2_imshow

# Upload image
uploaded = files.upload()

# Read image
img = cv2.imread("example.jpg")

# Display original image
print("Original Image")
cv2_imshow(img)

# -----
# 1. RESIZING
# -----

resized = cv2.resize(img, (500, 300))

print("Resized Image (500, 300)")
cv2_imshow(resized)

# -----
# 2. CROPPING
# -----

cropped = img[50:250, 50:250]

print("Cropped Image")
cv2_imshow(cropped)

# -----
# 3. ROTATION
# -----

# Get image dimensions
h, w = img.shape[:2]

# Find center of image
centre = (w // 2, h // 2)

# Create rotation matrix
matrix = cv2.getRotationMatrix2D(centre, 45, 1.0)

# Rotate image
rotated = cv2.warpAffine(img, matrix, (w, h))

print("Rotated Image (45 Degrees)")
cv2_imshow(rotated)
```

Input

Upload an image (the code reads **example.jpg**).

Output

Displays the original image, a 500 × 300 resized image, a cropped region, and the image rotated by 45 degrees.

Experiment-3: Image Filtering using Gaussian, Median and Bilateral Filters

Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload image
uploaded = files.upload()
filename = next(iter(uploaded))

# Read image in grayscale
i = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)

# Gaussian Filtering
g = cv2.GaussianBlur(i, (3, 3), sigmaX=1.0)

# Median Filtering
m = cv2.medianBlur(i, ksize=3)

# Bilateral Filtering
b = cv2.bilateralFilter(i, d=9, sigmaColor=75, sigmaSpace=75)

# Display Original Image
plt.figure(figsize=(15, 8))
plt.imshow(i, cmap='gray')
plt.title('Original Image')
plt.axis('off')
plt.show()

# Display Gaussian Filtered Image
plt.figure(figsize=(15, 8))
plt.imshow(g, cmap='gray')
plt.title('Gaussian Filtered Image')
plt.axis('off')
plt.show()

# Display Median Filtered Image
plt.figure(figsize=(15, 8))
plt.imshow(m, cmap='gray')
plt.title('Median Filtered Image')
plt.axis('off')
plt.show()

# Display Bilateral Filtered Image
plt.figure(figsize=(15, 8))
plt.imshow(b, cmap='gray')
plt.title('Bilateral Filtering')
plt.axis('off')
plt.show()
```

Input

Upload an image through Google Colab.

Output

Displays the original grayscale image and the Gaussian, Median and Bilateral filtered versions.

Experiment-5: Face Detection using Haar Cascade

Program

```
import cv2
from google.colab import files
from google.colab.patches import cv2_imshow

# Upload image
uploaded = files.upload()

# Get uploaded file name
filename = next(iter(uploaded))

# Read image
img = cv2.imread(filename)

# Check whether image was read successfully
if img is None:
    print("Error: Could not read the image.")
else:
    # Load Haar Cascade classifier
    face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + "haarcascade_frontalface_default.xml")

    # Convert image to grayscale
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Detect faces
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.5, minNeighbors=5)

    # Draw rectangles
    for (x, y, w, h) in faces:
        cv2.rectangle(img, (x, y), (x + w, y + h), (255, 0, 0), 2)

    # Display results
    print("Number of faces detected:", len(faces))
    cv2_imshow(img)
```

Input

Upload an image through Google Colab.

Output

Displays the image with rectangles around detected faces and prints the number of faces detected.

Experiment-7: POS Tagging and Named Entity Recognition (NER)

Program

```
import spacy

# Load English spaCy model
nlp = spacy.load("en_core_web_sm")

# Get input from user
text = input("Enter a sentence or paragraph: ")

print("\nOriginal Text:")
print(text)

# Process text
doc = nlp(text)

# POS TAGGING
print("\nPART-OF-SPEECH (POS) TAGGING")
for token in doc:
    print(f"{token.text:15} {token.pos_:10} {spacy.explain(token.pos_)}")

# NAMED ENTITY RECOGNITION (NER)
print("\nNAMED ENTITY RECOGNITION (NER)")
if len(doc.ents) == 0:
    print("No named entities found.")
else:
    for ent in doc.ents:
        print(f"{ent.text:20} {ent.label_:10} "
              f"{spacy.explain(ent.label_)}")
```

If using Google Colab
First run:
!pip install spacy
!python -m spacy download en_core_web_sm
Then run the main code above.

Input

RAMU EATS BANANA

Output

Input:
Enter a sentence or paragraph:
RAMU EATS BANANA

Output:
Original Text:
RAMU EATS BANANA

PART-OF-SPEECH (POS) TAGGING

RAMU	PROPN	proper noun
EATS	PROPN	proper noun
BANANA	PROPN	proper noun

NAMED ENTITY RECOGNITION (NER)
No named entities found.